

Peer-to-Peer File Distribution

❖ ১ ধারণা: Client-Server বনাম Peer-to-Peer (P2P)

Client-Server Architecture:

এই মডেলে সবকিছু একটা **কেন্দ্রীয় সার্ভার** এর মাধ্যমে হয়।

- সার্ভার সব ক্লায়েন্টকে (user/peer) ডাটা পাঠায়।
- ক্লায়েন্টরা নিজেরা একে অপরের সাথে ডাটা আদান-প্রদান করতে পারে না।
- সার্ভার সব কাজের “বোঝা” বহন করে।

উদাহরণ: ওয়েব, ই-মেইল, DNS ইত্যাদি।

📦 সমস্যা:

যদি অনেক ক্লায়েন্ট একসাথে কোনো বড় ফাইল চায় (যেমন 1 GB OS update), তবে সার্ভারকে N বার একই ফাইল পাঠাতে হয় → **Server bandwidth ও সময় দুই-ই অনেক লাগে।**

Peer-to-Peer (P2P) Architecture:

- এখানে সার্ভার কেবল শুরুতে ফাইলটা দেয়।
- এরপর যারা ফাইল পায় (peers), তারা নিজেরাই অন্য peer-দের ফাইলের অংশ পাঠাতে শুরু করে।
- অর্থাৎ সবাই একসাথে **upload + download** করতে থাকে।
- সার্ভারের ওপর চাপ অনেক কমে যায়।

📦 Peers মানে হলো — user দের PC, laptop, smartphone ইত্যাদি — যারা **নিজেরা control করে**, কোনো কোম্পানির না।

❖ ২ P2P File Distribution এর উদাহরণ — BitTorrent

👉 BitTorrent হলো সবচেয়ে **জনপ্রিয় P2P ফাইল শেয়ারিং প্রোটোকল।**

- বানিয়েছেন **Bram Cohen**।
- এক সার্ভার থেকে “seed” করা হয়, তারপর সবাই “peer” হয়ে ফাইলের টুকরো টুকরো অংশ (chunks) একে অপরকে পাঠায়।
- এতে সিস্টেম **self-scalable** হয় — যত peer বাড়ে, ততই বিতরণ দ্রুত হয়, কারণ সবাই মিলে শেয়ার করে।

◆ ৩ বিশ্লেষণ: Scalability (বিস্তার ক্ষমতা)

এখন আমরা তুলনা করবো:

- Client-Server আর্কিটেকচার
- Peer-to-Peer আর্কিটেকচার

একই ফাইল N জন peer কে পাঠাতে হবে।

ধরা যাক:

- ফাইলের আকার = F bits
- সার্ভারের upload rate = u_s
- i -তম peer এর upload rate = u_i
- i -তম peer এর download rate = d_i
- সবচেয়ে কম download speed যেই peer-এর, সেটাকে d_{min} বলা হচ্ছে
 $\Rightarrow d_{min} = \min(d_1, d_2, \dots, d_N)$

আমরা ধরে নিচ্ছি — Internet core এর bandwidth যথেষ্ট, bottleneck শুধু access link এ।

◆ ৪ Client-Server Distribution Time (D_{cs})

Client-server এ সার্ভারই একা ফাইলটা সবাইকে পাঠায়।

✚ দুটি শর্ত:

1. সার্ভারকে N জনের জন্য পুরো ফাইল পাঠাতে হবে $\rightarrow$ সময় লাগবে অন্ততঃ

$$\frac{NF}{u_s}$$

কারণ সার্ভারকে মোট NF bits পাঠাতে হয়, rate হলো u_s ।

2. সবচেয়ে ধীর peer ফাইলটা ডাউনলোড করতে পারবে না

$$\frac{F}{d_{min}}$$

সময়ের কমে।

👉 তাই মোট সময় হবে এই দুটির মধ্যে বড়টি:

$$D_{cs} = \max \left(\frac{NF}{u_s}, \frac{F}{d_{min}} \right)$$

● অর্থাৎ সার্ভার যত peer বাড়ে, তত সময় বাড়ে।

যেমন, যদি peer ১০০০ থেকে বেড়ে ১০ লক্ষ হয়, সময়ও প্রায় ১০০০ গুণ বেড়ে যাবে।

◆ P2P Distribution Time (Dp2p)

এবার ধরো, সবাই মিলে কাজ করছে (P2P আর্কিটেকচার)।

✚ তিনটি গুরুত্বপূর্ণ বিষয়:

1. শুরুতে শুধু সার্ভারের কাছে ফাইল আছে।
তাই অন্ততঃ সার্ভারকে একবার পুরো ফাইলটা আপলোড করতে হবে।
সময় $\geq F/u_s$
2. সবচেয়ে ধীর peer-এর ডাউনলোড করতে সময় লাগবে অন্ততঃ F/d_{min}
3. সিস্টেমের মোট upload ক্ষমতা হলো:

$$u_{total} = u_s + u_1 + u_2 + \dots + u_N$$

পুরো সিস্টেমকে N জন peer কে F bits করে দিতে হবে $\Rightarrow$ মোট ডাটা = NF bits
তাই সময় অন্ততঃ

$$\frac{NF}{u_{total}} = \frac{NF}{u_s + \sum_{i=1}^N u_i}$$

◆ এখন তিনটি একসাথে নিলে:

$$D_{P2P} = \max \left(\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum_{i=1}^N u_i} \right)$$

◆ তুলনামূলক বিশ্লেষণ:

◆ তুলনামূলক বিশ্লেষণ:

দিক	Client-Server	P2P	
কে আপলোড করে	শুধু সার্ভার	সার্ভার + সব peers	
Server load	অনেক বেশি	কম	
Scalability	সময় N এর সাথে লিনিয়ারলি বাড়ে	Self-scalable (N বাড়লে peers এর upload capacity বাড়ে)	
Distribution Time	$D_{cs} = \max \left(\frac{NF}{u_s}, \frac{F}{d_{min}} \right)$	$D_{P2P} = \max \left(\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum u_i} \right)$	

◆ Self-Scalability ব্যাখ্যা:

P2P সিস্টেমে যত peer বাড়ে, তত মোট upload ক্ষমতা বাড়ে।

অর্থাৎ ফাইল বিতরণে সবাই সাহায্য করে।

তাই “load” বাড়লেও সিস্টেম দ্রুত কাজ করতে পারে।

👉 এটাকেই বলা হয় self-scalability।

Client-server এ বিপরীত — peer বাড়লে সার্ভারের ওপর চাপও বাড়ে → সময় বাড়ে।

◆ সংক্ষেপে উদাহরণ:

◆ ধ্রুসংক্ষেপে উদাহরণ:

ধরা যাক:

- ফাইল = 15 GB
- $u_s = 30$ Mbps
- প্রতিটি peer $u_i = 10$ Mbps
- $N = 1000$ জন

তাহলে client-server এ:

$$D_{cs} = \frac{NF}{u_s} = \frac{1000F}{30} \rightarrow \text{অনেক বেশি সময়!}$$

কিন্তু P2P এ:

$$D_{P2P} = \frac{NF}{u_s + \sum u_i} = \frac{1000F}{30 + 1000 \times 10} = \frac{1000F}{10030} \approx 0.1F$$

অর্থাৎ সময় ১০ গুণ কমে গেলো।



Figure 2.23 ◆ Distribution time for P2P and client-server architectures

◆ ৯ সারসংক্ষেপ:

পয়েন্ট	ব্যাখ্যা
P2P এর ধারণা	Peers একে অপরকে ফাইল শেয়ার করে
লক্ষ্য	সার্ভারের ওপর চাপ কমানো
মূল সমীকরণ	$(D_{P2P} = \max(\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum u_i}))$
সুবিধা	Self-scalable, efficient, cost-effective
উদাহরণ	BitTorrent
তুলনা	P2P অনেক দ্রুত ও scalable client-server এর তুলনায়

◆ ১০ BitTorrent কী?

BitTorrent হচ্ছে এমন একটি P2P প্রোটোকল যেটি বড় ফাইল (যেমন গেম, মুভি, সফটওয়্যার, OS আপডেট ইত্যাদি) অনেক মানুষের মধ্যে দ্রুত ভাগাভাগি করার জন্য ব্যবহৃত হয়।

👉 মূল ধারণা:

সব peer (user) মিলে ফাইলের অংশ একে অপরকে দেয়।

- যখন তুমি ফাইল ডাউনলোড করছো, তখন একই সময়ে তুমি অন্যদেরও ফাইল আপলোড করছো।
- সবাই মিলেই সিস্টেমটাকে দ্রুত করে তুলছে।

◆ ১১ কিছু গুরুত্বপূর্ণ টার্ম

টার্ম	মানে
Torrent	একটি নির্দিষ্ট ফাইল শেয়ার করার জন্য অংশগ্রহণ করা peer দেব পুরো গ্রুপ
Peer	ফাইল শেয়ার করা যেকোনো কম্পিউটার বা ইউজার

টার্ম	মানে
Tracker	একটি সার্ভার যেটি peers দেব তালিকা রাখে (কে কে অনলাইনে আছে)
Chunk	ফাইলের সমান আকারের ছোট ছোট টুকরা (সাধারণত 256 KB প্রতি chunk)
Seeder	যে peer এর কাছে পুরো ফাইল আছে
Leecher	যে peer এখনো পুরো ফাইল পায়নি (ডাউনলোড করছে)

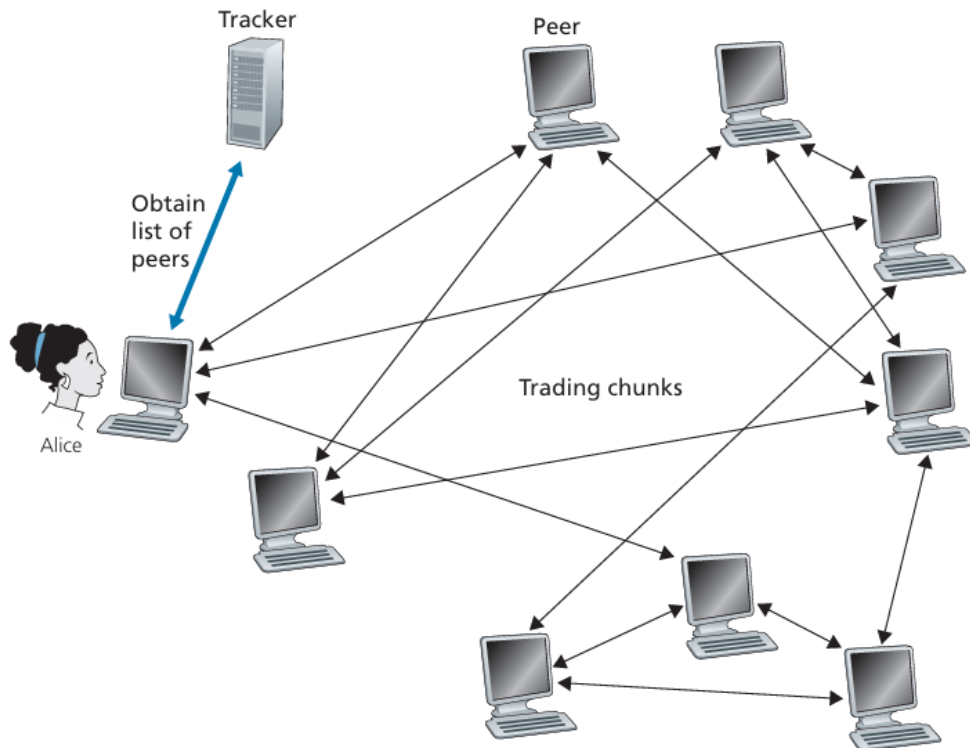


Figure 2.24 ♦ File distribution with BitTorrent

◆ ক্রিভাবে BitTorrent কাজ করে — ধাপে ধাপে

■ ধাপ ১: Peer যোগ দেওয়া

যখন নতুন কোনো peer (ধরা যাক Alice) কোনো টরেন্টে যোগ দেয়,

1. সে প্রথমে **tracker সার্ভারে রেজিস্টার করে** জানায় যে সে অংশগ্রহণ করতে চায়।
2. ট্র্যাকার তখন Alice কে কিছু (যেমন ৫০টি) peer এর IP address পাঠায়।

3. Alice ঐ peer গুলোর সাথে TCP কানেকশন তৈরি করার চেষ্টা করে।
— যাদের সাথে কানেকশন সফল হয়, তাদের বলে **neighboring peers**।

- ♦ সময়ের সাথে কিছু peer চলে যায়, **নতুন peer আসে** — তাই **neighbor list পরিবর্তন হয়**।

■ ধাপ ২: Chunk Information শেয়ার করা

- প্রতিটি peer এর কাছে ফাইলের কিছু নির্দিষ্ট chunk থাকে।
- Alice তার প্রত্যেক neighbor কে জিজ্ঞেস করে, “তোমার কাছে কোন কোন chunk আছে?”
- এতে Alice জানতে পারে — কার কাছে কী আছে।
- তারপর Alice সিদ্ধান্ত নেয়, সে কোন chunk আগে ডাউনলোড করবে।

■ ধাপ ৩: কোন chunk আগে নামাবে — **Rarest First Policy**

Alice তার কাছে নেই এমন chunk গুলোর মধ্যে **দেখে কোন chunk সবচেয়ে rare** — অর্থাৎ, তার আশেপাশের peers দের মধ্যে সবচেয়ে কম কপি আছে কোন chunk-এর।

👉 সেটাই আগে ডাউনলোড করে।

- ♦ এতে করে সব chunk প্রায় সমানভাবে নেটওয়ার্কে ছড়িয়ে পড়ে।
- ♦ কোনো chunk খুব কম পাওয়া যাচ্ছে — এমন সমস্যা হয় না।

■ ধাপ ৪: কাকে ফাইল পাঠাবে — **Tit-for-Tat Policy**

এটা BitTorrent এর সবচেয়ে বুদ্ধিমান অংশ — **ট্রেডিং বা incentive mechanism**।

Alice **দেখে কোন কোন peer তাকে সবচেয়ে দ্রুত ডাটা পাঠাচ্ছে** (সর্বোচ্চ rate এ)।

- সে তাদের মধ্যে **top 4 peer বেছে নেয়**, এবং শুধু তাদেরই chunk পাঠায়।
- এই ৪ জনকে বলা হয় **unchoked peers**।
- প্রতি ১০ সেকেন্ড পরপর এই তালিকা আপডেট হয়।

🕒 প্রতি ৩০ সেকেন্ড পর, Alice এলোমেলোভাবে আরেকজন নতুন peer (ধরা যাক Bob) কে বেছে নেয় এবং তাকে chunk পাঠায়।

- এই peer কে বলা হয় **optimistically unchoked peer**।

যদি Bob-ও ভালোভাবে ডাটা দেয়, তাহলে পরের রাউন্ডে সে top 4 এর মধ্যে চলে আসে।
এইভাবে নতুন peer রাও সুযোগ পায় অংশগ্রহণের এবং ট্রেডিং শুরু করতে পারে।

🔄 এর ফলে দ্রুত peer গুলো একে অপরকে খুঁজে নেয় যারা upload rate এর দিক থেকে মিল আছে —

যারা দ্রুত দিতে পারে, তারা দ্রুত পায়।

আর ধীর peer গুলোও ধীরে ধীরে নেটওয়ার্কে ঢুকে পড়ে।

■ ধাপ ৫: চোক এবং আনচোক

অবস্থা	ব্যাখ্যা
Unchoked peer	যাদেরকে Alice বর্তমানে ফাইল পাঠাচ্ছে
Optimistically unchoked peer	এলোমেলোভাবে নির্বাচিত peer, যার সাথে ট্রেড শুরু হচ্ছে
Choked peer	যাদেরকে Alice বর্তমানে কিছু পাঠাচ্ছে না

◆ ৪ Tit-for-Tat Mechanism (পারস্পরিক সুবিধা নীতি)

এটা একটা বুদ্ধিমান “give and take” সিস্টেম।

👉 তুমি যত দ্রুত আমাকে দাও, আমি তত দ্রুত তোমাকে দেবো।

এই নীতি ব্যবহার করার ফলে:

- সবাইকে আপলোডে অংশ নিতে হয়।
- কেউ free rider (শুধু ডাউনলোড করে, আপলোড না করে) হতে পারে না।
- নেটওয়ার্কে ভারসাম্য বজায় থাকে।

💡 যদি এই নীতি না থাকত, তাহলে সবাই শুধু ডাউনলোড করত — কেউ কিছু আপলোড করত না — ফলে সিস্টেম ভেঙে পড়ত।

◆ ৫ অন্যান্য অতিরিক্ত মেকানিজম (সংক্ষেপে)

মেকানিজম	কাজ
Pieces (mini-chunks)	প্রতিটি chunk আবার ছোট ছোট অংশে ভাগ করা হয় দ্রুত ট্রান্সফারের জন্য
Pipelining	একাধিক chunk এর রিকোয়েস্ট একসাথে পাঠানো হয় — throughput বাড়ায়
Random First Selection	শুরুতে র্যান্ডম chunk বেছে নেওয়া হয় দ্রুত শুরু করতে
Endgame Mode	ডাউনলোড শেষের দিকে একই chunk একাধিক peer থেকে নেওয়া হয় — শেষ করতে দেরি না হয়
Anti-snubbing	যেসব peer ধীরে পাঠায় তাদের এড়িয়ে চলা হয়

◆ ৬ Distributed Hash Table (DHT)

DHT হলো P2P নেটওয়ার্কে ব্যবহৃত একটি distributed database system।

- এখানে ডাটার “key-value pair” গুলো peer দেব মধ্যে ভাগ করে রাখা হয়।
- কোনো peer ডাটা সার্চ করলে, DHT ব্যবহার করে সেটি কোন peer এর কাছে আছে তা খুঁজে বের করা হয়।

📌 BitTorrent এর আধুনিক সংস্করণগুলোতে DHT ব্যবহার করা হয় tracker ছাড়া peer খুঁজে পেতে।

◆ ৭ সারসংক্ষেপ

বিষয়	ব্যাখ্যা
প্রোটোকল নাম	BitTorrent
প্রধান কাজ	ফাইলের ছোট ছোট অংশ peer-দের মধ্যে বিতরণ করা
Chunk আকার	সাধারণত 256 KB
Tracker এর কাজ	সক্রিয় peer-দের তালিকা রাখা
ডাউনলোড নীতি	Rarest First
আপলোড নীতি	Tit-for-Tat (Top 4 + 1 random peer)

বিষয়	ব্যাখ্যা
সুবিধা	Self-scalable, fast, efficient
ঝুঁকি	কিছু peer free rider হতে পারে (তবে tit-for-tat সেটা নিয়ন্ত্রণ করে)

2.6 Video Streaming and Content Distribution Networks (CDNs)

আজকের দিনে ইন্টারনেটের ট্র্যাফিকের প্রায় ৮০% ভিডিও স্ট্রিমিং সার্ভিস (যেমন Netflix, YouTube, Amazon Prime) দ্বারা দখল করা হয়। এই অংশে বোঝানো হয়েছে—ভিডিও স্ট্রিমিং কীভাবে কাজ করে, এবং DASH (Dynamic Adaptive Streaming over HTTP) কীভাবে ভিডিও স্ট্রিমিংকে আরও কার্যকর করেছে।

2.6.1 Internet Video

ভিডিও কীভাবে কাজ করে

একটি ভিডিও হলো একাধিক ইমেজ বা ফ্রেমের একটি সিরিজ, যা সাধারণত প্রতি সেকেন্ডে ২৪ বা ৩০টি ইমেজ দেখানো হয়। প্রতিটি ফ্রেম পিক্সেল দিয়ে তৈরি এবং প্রতিটি পিক্সেল রঙ ও উজ্জ্বলতার মান বহন করে।

কম্প্রেশন(Compression)

ভিডিওর সাইজ কমাতে কম্প্রেশন অ্যালগরিদম ব্যবহার করা হয়। এতে ভিডিওর মান কিছুটা কমলেও ডেটার পরিমাণ অনেক কমে যায়।

উদাহরণ:

- **Low quality:** 100 kbps
- **HD movie:** 4 Mbps
- **4K video:** 10 Mbps+

যেমন, ২ Mbps গতির একটি ৬৭ মিনিটের ভিডিও প্রায় ১ GB স্টোরেজ নেবে।

Throughput (ডেটা ট্রান্সফার গতি)

ভিডিও যেন বিরতি ছাড়া দেখা যায়, সে জন্য নেটওয়ার্কের গড় throughput অবশ্যই ভিডিওর bitrate-এর সমান বা বেশি হতে হবে।

Multiple Quality Levels

একই ভিডিওর বিভিন্ন গুণগত মান (যেমন 300 kbps, 1 Mbps, 3 Mbps) একাধিক সংস্করণ তৈরি করা যায়।

ব্যবহারকারী তাদের ইন্টারনেট গতির ওপর ভিত্তি করে যে সংস্করণটি সুবিধাজনক সেটি দেখতে পারে।

◆ 2.6.2 HTTP Streaming এবং DASH

HTTP Streaming

- ভিডিও সার্ভারে সাধারণ ফাইল আকারে সংরক্ষিত থাকে।
- ব্যবহারকারী যখন দেখতে চায়, তখন **HTTP GET request** পাঠায়।
- সার্ভার **TCP connection** এর মাধ্যমে ভিডিও ফাইল পাঠায়।
- ক্লায়েন্ট পাশে একটি **buffer** এ ভিডিও জমা হয় এবং কিছুটা জমার পর থেকেই ভিডিও প্লে শুরু হয়।

সমস্যা:

সব ব্যবহারকারী একই ভিডিও কোয়ালিটি পায়, যদিও সবার ব্যান্ডউইথ এক নয়।

DASH (Dynamic Adaptive Streaming over HTTP)

কীভাবে কাজ করে

- একই ভিডিওর একাধিক **bitrate** সংস্করণ থাকে (যেমন 300 kbps, 1 Mbps, 3 Mbps)।
- ভিডিওটি ছোট ছোট **chunks (২-১০ সেকেন্ড)** এ ভাগ করা হয়।
- ক্লায়েন্ট প্রতি **chunk** আলাদা করে **HTTP GET** রিকোয়েস্টের মাধ্যমে ডাউনলোড করে।

Adaptation

- ক্লায়েন্ট তার ব্যান্ডউইথ অনুযায়ী কোন সংস্করণ (low/high bitrate) নেবে তা নির্ধারণ করে।
- যদি ব্যান্ডউইথ বাড়ে → **উচ্চ মানের chunk নেয়**
- যদি ব্যান্ডউইথ কমে → **নিম্ন মানের chunk নেয়**

 সার্ভারে একটি **manifest file** থাকে যেখানে ভিডিওর সব সংস্করণের **URL ও bitrate তথ্য** দেওয়া থাকে।

ক্লায়েন্ট প্রথমে manifest ফাইলটি ডাউনলোড করে এবং এরপর প্রতিটি chunk অনুরোধ করে।

☑ DASH-এর সুবিধা

1. বিভিন্ন ইন্টারনেট গতির জন্য উপযুক্ত ভিডিও মান নির্বাচন সম্ভব।
2. ব্যান্ডউইথ পরিবর্তন হলেও ভিডিও বন্ধ না হয়ে স্বয়ংক্রিয়ভাবে মান কমানো/বাড়ানো হয়।
3. মোবাইল ব্যবহারকারীদের জন্য আদর্শ, কারণ তাদের নেটওয়ার্ক গতি প্রায়ই ওঠানামা করে।

🌐 2.6.3 Content Distribution Networks (CDN)

🎯 মূল উদ্দেশ্য:

বিশ্বজুড়ে কোটি কোটি ব্যবহারকারীর কাছে ভিডিও (যেমন YouTube, Netflix, Amazon Prime Video ইত্যাদি) দ্রুত, নিরবচ্ছিন্ন এবং মানসম্পন্নভাবে পৌঁছে দেওয়া।

⚠️ **যদি সব ভিডিও এক ডেটা সেন্টার থেকে পাঠানো হয় তাহলে তিনটি বড় সমস্যা হয়:**

1. **দূরত্ব সমস্যা:**
যদি ব্যবহারকারী ডেটা সেন্টার থেকে অনেক দূরে থাকে, তাহলে ডেটা অনেক রাউটার ও ISP-এর মধ্য দিয়ে যেতে হয় →
📉 **থ্রুপুট (throughput) কমে** → ভিডিও **বাফারিং বা ফ্রিজিং** হয়।
2. **একই ডেটা বারবার পাঠানো:**
জনপ্রিয় ভিডিও বারবার একই পথে পাঠানো হয় →
📦 **ব্যান্ডউইথ নষ্ট হয়** এবং
💰 **কোম্পানিকে বেশি খরচ করতে হয়।**
3. **একটি পয়েন্টে সমস্যা মানেই পুরো সিস্টেম বন্ধ:**
একমাত্র ডেটা সেন্টার নষ্ট হলে →
❌ **সব ভিডিও বন্ধ হয়ে যাবে।**



সমাধান: CDN (Content Distribution Network)

CDN হলো বিশ্বজুড়ে ছড়িয়ে থাকা সার্ভার ক্লাস্টারের একটি নেটওয়ার্ক যা ভিডিও, ছবি, অডিও, ওয়েব পেজ ইত্যাদি কনটেন্ট ব্যবহারকারীর কাছে সার্ভার থেকে সরবরাহ করে।

👉 এর ফলে

- কম লেটেন্সি (delay)
- বেশি গতি (throughput)
- কম লোড ও খরচ পাওয়া যায়।

🏠 CDN-এর ধরন:

1. **Private CDN:**
কোম্পানির নিজস্ব CDN।
উদাহরণ:
 - Google-এর CDN (YouTube-এর জন্য)
 - Netflix Open Connect
2. **Third-party CDN:**
একাধিক কোম্পানির জন্য কনটেন্ট সরবরাহ করে।
উদাহরণ:
 - Akamai
 - Limelight
 - Level-3

⚙️ CDN Server Placement Philosophy (দুই ধরনের স্থাপন কৌশল):

◆ 1 Enter Deep (Akamai Model)

👉 এটা Akamai Technologies-এর পদ্ধতি।

🔧 কীভাবে কাজ করে:

- ছোট ছোট সার্ভার ক্লাস্টার (cache servers) সরাসরি ISP (Internet Service Provider)-এর নেটওয়ার্কের ভিতরে বসানো হয়।

- মানে, ইউজার যেই ইন্টারনেট সার্ভিস ব্যবহার করছে, সেই ISP-র ভেতরেই একটা Akamai সার্ভার থাকে।
- ফলে ডেটা খুব ছোট দূরত্ব অতিক্রম করে ইউজারের কাছে পৌঁছে যায়।

☀ সুবিধা:

- ☑ ব্যবহারকারীর কাছাকাছি সার্ভার →
ডেটা ট্রান্সফার সময় কম (low latency), গতি বেশি (high throughput)
- ☑ ব্যবহারকারীর অবস্থান অনুযায়ী কন্টেন্ট সবচেয়ে কাছের সার্ভার থেকে পাওয়া যায়।

⚠ অসুবিধা:

- ✗ হাজার হাজার ছোট সার্ভার স্থাপন করতে হয় →
রক্ষণাবেক্ষণ (maintenance), আপডেট, মনিটরিং — সবকিছুই জটিল ও ব্যয়বহুল।
- উদাহরণ: Akamai CDN বিশ্বজুড়ে ৪,০০০+ লোকেশনে সার্ভার ক্লাস্টার স্থাপন করেছে।

◆ 2 Bring Home (Limelight Model)

👉 এটা Limelight Networks-এর পদ্ধতি।

🔧 কীভাবে কাজ করে:

- এখানে ছোট ছোট অনেক সার্ভার না বসিয়ে, কয়েকটা বড় সার্ভার ক্লাস্টার তৈরি করা হয়।
- এই বড় ক্লাস্টারগুলো সাধারণত IXP (Internet Exchange Points)-এ বসানো হয় —
যেখানে অনেক ISP একসাথে যুক্ত থাকে।
- ফলে এক জায়গার সার্ভার থেকেই অনেক ব্যবহারকারীকে সার্ভিস দেওয়া যায়।

☀ সুবিধা:

- ☑ কম সংখ্যক সার্ভার →
সহজ রক্ষণাবেক্ষণ, পরিচালনা ও আপডেট করা সহজ।
- ☑ বড় সার্ভার ক্লাস্টার → শক্তিশালী হার্ডওয়্যার, বেশি ক্যাশ ক্ষমতা।

⚠ অসুবিধা:

- ✗ ব্যবহারকারীর থেকে দূরে থাকে →
কখনও কখনও লেটেন্সি একটু বেশি হয় (বিশেষত দূরের গ্রাহকদের জন্য)।

- উদাহরণ: Limelight-এর প্রায় ৫০টা বড় ক্লাস্টার আছে বিশ্বজুড়ে গুরুত্বপূর্ণ IXP পয়েন্টে।

Content Replication (ভিডিও কপি করা):

- CDN সব ক্লাস্টারে সব ভিডিও রাখে না।
- জনপ্রিয় বা নির্দিষ্ট অঞ্চলের ভিডিওগুলোই বেশি রাখা হয়।
- সাধারণত **pull strategy** ব্যবহার হয়:
 - যদি কোনো ক্লাস্টারে ভিডিও না থাকে, তাহলে সেটি **central server বা অন্য ক্লাস্টার থেকে টেনে নেয়** এবং একই সঙ্গে ব্যবহারকারীর কাছে পাঠায়।
 - পরে ভিডিওটি লোকালভাবে **cache করে রাখে**।
 - যদি জায়গা পূর্ণ হয়, তাহলে **কম ব্যবহৃত ভিডিও মুছে ফেলে**।

CDN Operation (কাজের ধাপ)

চলুন একটি উদাহরণে দেখি 🖱️

ধরো, **NetCinema** নামের একটি কোম্পানি তার ভিডিও সার্ভ করার জন্য **KingCDN** ব্যবহার করছে।

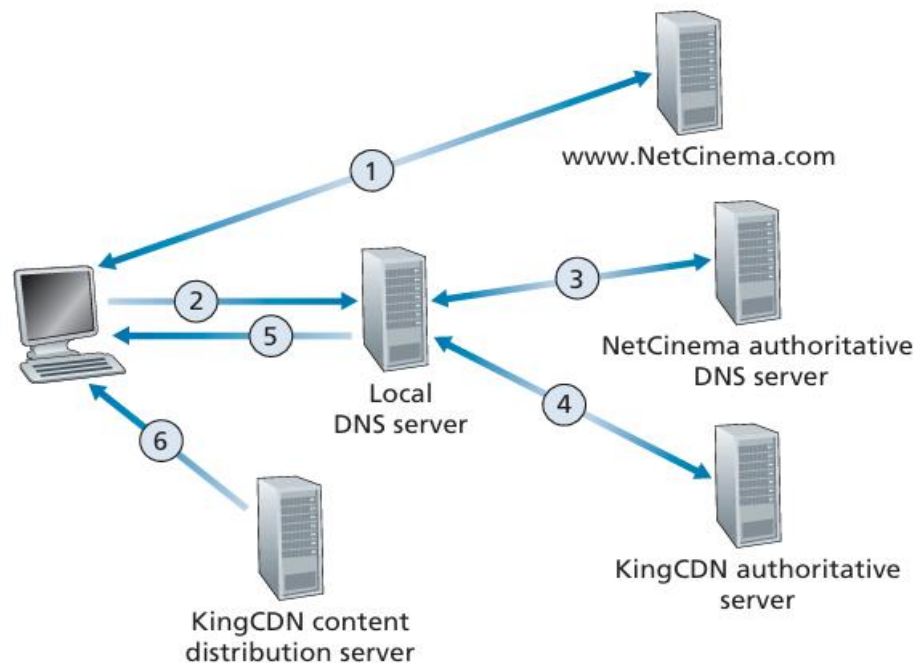


Figure 2.25 ♦ DNS redirects a user's request to a CDN server

ধাপে ধাপে প্রক্রিয়া:

1. ব্যবহারকারী NetCinema-এর ওয়েব পেজে যায়।
2. যখন সে ভিডিও লিঙ্কে ক্লিক করে → <http://video.netcinema.com/6Y7B23V>, তখন তার কম্পিউটার DNS query পাঠায় video.netcinema.com-এর জন্য।
3. NetCinema-এর authoritative DNS server দেখে যে এটি ভিডিও ডোমেইন → এটি রিকোয়েস্টটি KingCDN-এ পাঠিয়ে দেয় এবং রিটার্ন করে a1105.kingcdn.com নামটি।
4. এখন রিকোয়েস্ট KingCDN-এর DNS সিস্টেমে প্রবেশ করে। KingCDN ক্লায়েন্টের অবস্থান অনুযায়ী একটি উপযুক্ত সার্ভারের IP address দেয়।
5. এই IP address ব্যবহারকারীর কম্পিউটারে ফিরে আসে।
6. ব্যবহারকারী সেই সার্ভারে সরাসরি TCP কানেকশন স্থাপন করে এবং ভিডিওর জন্য HTTP GET request পাঠায়।
(যদি DASH ব্যবহার হয়, আগে একটি manifest file পাঠানো হয়।)

Cluster Selection Strategy (কোন সার্ভার থেকে পাঠানো হবে?)

CDN-কে নির্ধারণ করতে হয় কোন সার্ভার ক্লাস্টার থেকে ব্যবহারকারীকে ভিডিও দেওয়া হবে। এটি নির্ভর করে ব্যবহারকারীর Local DNS Server (LDNS) এর IP থেকে পাওয়া অবস্থানের উপর।

কিছু জনপ্রিয় পদ্ধতি:

1. **Geographic Proximity:**
ব্যবহারকারীর LDNS অবস্থান দেখে নিকটবর্তী সার্ভার নির্বাচন করা হয়।
✓ সহজ এবং দ্রুত
✗ কিন্তু কখনও কখনও LDNS ব্যবহারকারীর থেকে দূরে থাকতে পারে → ভুল সার্ভার নির্বাচিত হয়।
2. **Real-Time Performance Measurement:**
CDN তার সার্ভারগুলো থেকে নিয়মিত ping / probe message পাঠিয়ে বিভিন্ন LDNS-এর সাথে delay, loss rate, throughput পরিমাপ করে। তারপর সবচেয়ে ভালো পারফর্ম করা ক্লাস্টার বেছে নেয়।
✗ তবে সব LDNS probe-এর জবাব দেয় না, তাই এটি সবসময় কাজ নাও করতে পারে।

সারাংশ:

বিষয়	ব্যাখ্যা
CDN-এর উদ্দেশ্য	ব্যবহারকারীর কাছাকাছি সার্ভার থেকে কনটেন্ট দেওয়া
দুই ধরনের ডিজাইন	Enter Deep (Akamai), Bring Home (Limelight)
কনটেন্ট কপি করার পদ্ধতি	Pull strategy + caching
রিকোয়েস্ট হ্যান্ডলিং	DNS-based redirection
ক্লাস্টার নির্বাচন কৌশল	Geo-location বা Real-time Performance ভিত্তিক

◆ Netflix-এর মূল দুইটি অংশ

Netflix-এর ভিডিও ডেলিভারি সিস্টেম দুটি বড় ভাগে ভাগ করা যায়:

1. Amazon Cloud Infrastructure
2. Netflix-এর নিজস্ব CDN (Content Delivery Network)

🟡 1. Amazon Cloud-এর কাজসমূহ

Netflix-এর ওয়েবসাইট এবং অনেক ব্যাকএন্ড কাজ Amazon Web Services (AWS)-এ চলে।

এখানে Amazon Cloud-এর কাজগুলো হলো:

- **User management:** রেজিস্ট্রেশন, লগইন, বিলিং, ব্রাউজিং, সার্চ, রিকমেন্ডেশন সিস্টেম — সব ওয়েব ফাংশন এখানে চলে।
- **Content ingestion:** স্টুডিও থেকে মূল (master) মুভি ফাইল Netflix প্রথমে Amazon ক্লাউডে আপলোড করে।
- **Content processing:** এই মুভিগুলো বিভিন্ন ফরম্যাট ও বিটরেটে কনভার্ট করা হয় — যেমন মোবাইল, ডেস্কটপ, টিভি বা গেম কনসোলের জন্য আলাদা সংস্করণ।
- **Uploading to CDN:** প্রসেসিং শেষ হলে তৈরি হওয়া ভিডিও সংস্করণগুলো Netflix-এর CDN সার্ভারে আপলোড করা হয়।

🌐 2. Netflix Private CDN (Open Connect)

শুরুতে Netflix তিনটি third-party CDN ব্যবহার করত, কিন্তু পরে তারা নিজের CDN system তৈরি করে — যেটির নাম Netflix Open Connect।

এই CDN-এর বৈশিষ্ট্য:

- Netflix নিজেই সার্ভার র‍্যাক স্থাপন করে:
 - **IXP (Internet Exchange Point)**-এ
 - **Residential ISP (Internet Service Provider)**-এ
- বর্তমানে ২০০+ IXP এবং শত শত ISP লোকেশনে Netflix-এর র‍্যাক আছে।
- প্রতিটি র‍্যাকে:
 - কয়েকটি 10 Gbps Ethernet পোর্ট থাকে
 - 100 TB-এর বেশি স্টোরেজ থাকে

IXP র‍্যাকগুলোতে সাধারণত পুরো Netflix লাইব্রেরি থাকে (বিভিন্ন ভিডিও ভার্সনসহ)।

Push Caching System

Netflix **pull caching** না করে **push caching** ব্যবহার করে।

মানে — সার্ভারে **ভিডিও off-peak সময়ে (রাতের দিকে)** নিজে থেকেই পাঠানো হয়।

- যেসব সার্ভারে পুরো লাইব্রেরি রাখা সম্ভব না, সেখানে শুধু জনপ্রিয় ভিডিওগুলো পাঠানো হয় (প্রতিদিন আপডেট হয়)।

কিভাবে ভিডিও দেখা যায় (Client Interaction Process)

1. তুমি যখন Netflix-এ লগইন করে একটি মুভি সিলেক্ট করো — ওয়েবসাইটটি Amazon cloud-এ চলে, তাই তোমার রিকোয়েস্ট সেখানেই যায়।
2. Amazon ক্লাউড নির্ধারণ করে — কোন CDN সার্ভারে সেই মুভির কপি আছে।
3. “সেরা” সার্ভারটি বেছে নেয় (তোমার অবস্থান অনুযায়ী):
 - যদি তোমার ISP-তে Netflix সার্ভার থাকে → সেখান থেকে ভিডিও পাঠায়।
 - না থাকলে → নিকটবর্তী IXP সার্ভার থেকে পাঠায়।
4. তারপর Netflix তোমার ডিভাইসকে দেয়:
 - CDN সার্ভারের IP address
 - একটি **manifest file**, যেখানে ভিডিওর সব ভার্সনের URL থাকে।
5. এরপর তোমার ক্লায়েন্ট (Netflix অ্যাপ/ব্রাউজার) এবং সেই CDN সার্ভার সরাসরি **যোগাযোগ করে DASH প্রটোকল ব্যবহার করে ভিডিও স্ট্রিম করে।**

⚙ Adaptive Streaming

- ভিডিও ফাইল অনেকগুলো 4-second chunk-এ ভাগ করা থাকে।
- তোমার নেটওয়ার্ক স্পিড অনুযায়ী ক্লায়েন্ট পরের chunk-এর কোয়ালিটি বেছে নেয়।
- ফলে ভিডিও প্লে হয় smoothly without buffering।

🧠 Netflix CDN-এর বিশেষত্ব

- DNS redirect ব্যবহার করে না (অন্য CDN-দের মতো)।
- Netflix নিজেই নির্ধারণ করে কোন CDN সার্ভার ব্যবহার করবে।
- Push caching-এর কারণে cache miss হয় না।
- CDN শুধু ভিডিও সার্ভ করে — ওয়েব কন্টেন্ট নয়।

🌐 ISP (Internet Service Provider)

ISP মানে Internet Service Provider —
যে কোম্পানি বা সংস্থা ব্যবহারকারীকে ইন্টারনেট সংযোগ দেয়।

🔧 কাজ:

- ইন্টারনেট কানেকশন দেয় (ব্রডব্যান্ড, ফাইবার, মোবাইল ডেটা, ইত্যাদি)
- ইউজারদের IP ঠিকানা, DNS সার্ভার, রাউটিং, এবং ব্যান্ডউইথ সরবরাহ করে
- সাধারণত ঘর, অফিস, স্কুল, বিশ্ববিদ্যালয় বা কোম্পানিকে ইন্টারনেট দেয়

📶 বাংলাদেশে কিছু জনপ্রিয় ISP:

ধরন	নাম
বড় জাতীয় ISP	Link3 Technologies Ltd.
	Amber IT Ltd.
	Carnival Internet
	ICC Communication
	Triangle Services Ltd.

ধরন	নাম
	Agni Systems Ltd.
	IS Pros Ltd.
	BDCOM Online Ltd.
	Dot Internet
মোবাইল অপারেটর (ISP হিসাবেও কাজ করে)	Grameenphone, Robi, Banglalink, Teletalk

→ এরা সবাই মূলত ইন্টারনেট ইউজারদের সার্ভিস দেয় — তোমার বাসায় বা অফিসে যে ওয়াইফাই বা ব্রডব্যান্ড আসে, সেটা ISP-এর মাধ্যমেই আসে।

IXP (Internet Exchange Point)

IXP মানে Internet Exchange Point —

একটা ফিজিক্যাল নেটওয়ার্ক সেন্টার, যেখানে বিভিন্ন ISP, CDN, এবং বড় সার্ভার একে অপরের সঙ্গে সরাসরি ট্রাফিক আদান-প্রদান করে (peering)।

কাজ:

- দেশের ভেতরের ISP-রা যেন পরস্পরের ট্রাফিক লোকালি এক্সচেঞ্জ করতে পারে (বিদেশে ঘুরে না গিয়ে)
- এতে লেটেন্সি কমে, স্পিড বাড়ে, আর বিদেশি ব্যান্ডউইথ খরচ কমে
- অনেক সময় CDN সার্ভার (যেমন Google, Facebook, Akamai) এখানেই বসানো হয়

BD বাংলাদেশে প্রধান IXP সমূহ:

নাম	পরিচালন সংস্থা	অবস্থান
BDIX (Bangladesh Internet Exchange)	BDIX Foundation	ঢাকা (মূল কেন্দ্র), চট্টগ্রাম, সিলেট ইত্যাদি
NOVOC IXP	NovoCom Limited	ঢাকা
FICX (Fiber@Home Internet Exchange)	Fiber@Home Ltd.	ঢাকা

নাম	পরিচালন সংস্থা	অবস্থান
Amber IXP	Amber IT Ltd.	ঢাকা
Link3 IXP	Link3 Technologies Ltd.	ঢাকা

→ সবচেয়ে জনপ্রিয় এবং পুরনো IXP হলো BDIX, যেখানে প্রায় সব ISP যুক্ত আছে।
তুমি যেসব বাংলাদেশি ওয়েবসাইট দেখো (যেমন .bd ডোমেইন), অনেক সময় সেই ডেটা BDIX-এর মধ্যেই এক্সচেঞ্জ হয়।

সম্পর্ক (IXP vs ISP)

তুমি এভাবে ভাবতে পারো 🙌

```
[ Internet Backbone (World) ]
    ↓
  [ IXP ] ← এখানে সবাই যুক্ত হয়
    ↓ ↓ ↓
[ ISP1 ] [ ISP2 ] [ ISP3 ] → User (তুমি)
```

- IXP হলো “মিটিং পয়েন্ট”
- ISP হলো “তোমার সার্ভিস প্রোভাইডার”

IXP ছাড়া ISP-দের নিজেদের মধ্যে কানেকশন করতে হলে বিদেশে ট্রাফিক পাঠাতে হতো — এতে খরচ ও সময় দুটোই বেড়ে যেত।

YouTube Video Distribution System Explained

♦ সারসংক্ষেপ

YouTube (Google-এর মালিকানাধীন) পৃথিবীর সবচেয়ে বড় ভিডিও-শেয়ারিং সাইট, যেখানে প্রতি মিনিটে শত শত ঘন্টা ভিডিও আপলোড হয় এবং প্রতিদিন কোটি কোটি ভিডিও ভিউ হয়।

✱ YouTube Architecture: প্রধান উপাদান

YouTube-এর আর্কিটেকচারকে মূলত তিনটি ভাগে ভাগ করা যায়:

1. Google Data Centers
2. Google Private CDN (Content Delivery Network)
3. DNS-based Load Balancing and Caching System

☁ 1. Google Data Centers

- ভিডিও আপলোড হওয়ার পর প্রক্রিয়াকরণ (processing) সম্পূর্ণভাবে Google-এর ডেটা সেন্টারগুলোতে সম্পন্ন হয়।
- প্রতিটি ভিডিওকে YouTube নিজস্ব ফরম্যাটে রূপান্তর করে।
- একাধিক সংস্করণ (version) তৈরি হয় — প্রতিটি আলাদা bit rate ও quality level-এ।
- এই প্রস্তুত ভিডিও ফাইলগুলো পরে CDN সার্ভারগুলিতে পাঠানো হয়।

🌐 2. Google's Private CDN (YouTube CDN)

Netflix-এর মতোই, YouTube-ও নিজস্ব private CDN ব্যবহার করে, যা শত শত IXP ও ISP লোকেশনে সার্ভার ক্লাস্টার আকারে ছড়িয়ে আছে।

- YouTube CDN সার্ভারগুলোতে ভিডিও কন্টেন্ট রাখা হয়।
- ভিডিওগুলো Google Data Center থেকে এই সার্ভারগুলোতে pull caching পদ্ধতিতে আসে।

📦 Pull Caching System

Netflix push caching ব্যবহার করলেও, YouTube pull caching ব্যবহার করে।

মানে —

যখন কোনো ইউজার নতুন ভিডিও দেখতে চায় এবং সেটা ওই সার্ভারে না থাকে, তখন সার্ভার Google Data Center থেকে ভিডিও “টেনে আনে” (pull করে)।

এই ভিডিওটি একবার ক্যাশে আসার পর, পরবর্তী ইউজাররা একই সার্ভার থেকে সেটি দ্রুত দেখতে পারে।

3. DNS Redirect and Cluster Selection

YouTube ক্লায়েন্টকে কোন সার্ভার থেকে ভিডিও পাঠাবে সেটা নির্ধারণ করতে **DNS redirect** ব্যবহার করে।

প্রক্রিয়াটি এমনভাবে চলে:

1. ক্লায়েন্ট (তোমার ডিভাইস) যখন ভিডিও দেখতে চায়, DNS কুয়েরি করে।
2. Google-এর DNS সিস্টেম দেখে —
কোন সার্ভার ক্লাস্টারের RTT (Round Trip Time) সবচেয়ে কম।
3. সাধারণত ক্লায়েন্টকে সবচেয়ে কাছের সার্ভারে পাঠানো হয়, তবে **load balancing** বজায় রাখতে মাঝে মাঝে দূরের ক্লাস্টারে রিডাইরেক্ট করা হয়।

Streaming Mechanism

- YouTube **HTTP-based streaming** ব্যবহার করে।
- প্রতিটি **ভিডিওর জন্য কয়েকটি নির্দিষ্ট bitrate version** তৈরি করা হয়।
- Netflix-এর মতো **adaptive streaming (DASH)** ব্যবহার করে না।
- ইউজারকে **ম্যানুয়ালি ভিডিও কোয়ালিটি** (যেমন 480p, 720p, 1080p) নির্বাচন করতে হয়।

Byte Range Requests

YouTube HTTP byte range request ব্যবহার করে —

এটা এমন একটা মেকানিজম যা বলে দেয় “ভিডিওর কোন অংশটা পাঠাতে হবে”।

এর ফলে:

- ইউজার ভিডিও স্কিপ করলে অপ্রয়োজনীয় অংশ ট্রান্সমিট হয় না।
- ব্যান্ডউইথ ও সার্ভার রিসোর্স বাঁচে।
- ভিডিও দেখা আরও কার্যকর হয়।

Video Upload Process

- ভিডিও আপলোডও হয় HTTP প্রোটোকল দিয়ে।
- আপলোডের পর YouTube ভিডিওকে প্রসেস করে একাধিক ফরম্যাটে কনভার্ট করে।
- এই পুরো প্রসেসিং কাজ হয় Google Data Center-এর ভেতরেই।



Netflix বনাম YouTube — মূল পার্থক্য

বিষয়	Netflix	YouTube
CDN ধরন	Private CDN	Private CDN
Caching পদ্ধতি	Push Caching	Pull Caching
DNS Redirect ব্যবহার করে?	না	হ্যাঁ
Adaptive Streaming (DASH)	হ্যাঁ	না (Manual quality select)
Content Type	Licensed Movies & Series	User-uploaded Videos
Streaming Protocol	Proprietary DASH	HTTP Streaming
Load Balancing Method	Netflix software নির্ধারণ করে	DNS-based RTT এবং Load balancing
Video Chunking	4-second chunks	Byte range request